

ACRME Calculation Logic Reference — All Scenarios (v2.2)

Revision v2.2 — 27 August 2026. This document supersedes the previous calculation logic reference. Key changes in v2.2: - **DR sizing formula replaced:** fixed `dr_ratio_*` percentage approach (v1) is superseded by the **max-not-sum distributed-portion formula** (Requirements v2.1 Appendix A.6 / Appendix D). - **Reservation target floor formula added:** `Allocated VM Count + Configured Buffer (CAP-003)`, distinct from the forecast-based growth formula. - **Four new scenarios added:** DR destination requirement (max-not-sum), standby activation, deployment readiness gate, and customer seed record. - **EU cross-geo DR region corrected:** Belgium Central → **Switzerland North** throughout. - **Quota grouping model updated (QUA-004):** **single governed pool** per region/quota family (Prod + NonProd/CVAL + DR together) is now the **primary model**; Prod/DR protected by logical earmarks. The two-group topology is retained only as a narrow governance/Azure-limit exception (Scenario 8/9 updated accordingly). - **Scenario 15** (fixed DR ratio parameters) retained for reference but marked **superseded**.

This document consolidates **every calculation logic** used by the Azure Capacity Reservation Management Engine (ACRME), organized by the scenario in which it fires. Each formula carries an evidence tag:

- **[Documented]** — backed by Microsoft Learn or Azure platform documentation.
- **[Decided]** — an approved design decision recorded in the ADR Decision Log.
- **[Derived]** — a logical consequence of the design; requires POC validation where noted.
- **[Assumed]** — a policy default or working hypothesis, tunable and not yet empirically validated.

Sources: Requirements Baseline v2.1/v2.2 (Appendix A–D); ADR-001 through ADR-004 v1.2; `acrme_production_readiness_review_and_architecture.md` (PRR Section 26–Section 32).

Convention. `vCPU` = `vCPU_per_instance` for the SKU family. CVAL (Customer Validation) and NonProd are interchangeable; scoring identifiers keep the NonProd spelling for engineering consistency. All raw ratios are clamped with `Clamp(x) = max(0, min(1, x))` before weighting.

Index of Scenarios

#	Scenario	Core calculation	Status
1	Prod region derivation — customer picks a geography (exception path) ; explicit approval + customer acknowledgement required)	$\text{argmax}(\text{PS_Prod})$ over Standard regions	Current
2	Prod region validation — customer supplies the exact region (default input) ; ACRME validates, does not derive)	HC-1..HC-10 gate + PS_Prod post-validation	Current
3	Restricted region request	Exception workflow (no scoring)	Current
4	Middle East three-region deployment	$\text{argmax}(\text{PS_Prod})$ in-geo; DR currently NOT_OFFERED (DR-014, DEC-001 under legal review) — Switzerland North cross-geo DR is pre-configured but inactive , conditional on DEC-001	Updated
5	CVAL / NonProd region selection	$\text{argmax}(\text{PS_NonProd})$	Current
6	DR region selection	$\text{argmax}(\text{PS_DR})$	Current
7	Hard-constraint eligibility gate	HC-3, HC-6, HC-7 arithmetic	Current

#	Scenario	Core calculation	Status
8	Quota group sizing	Prod & NonProd+DR group budgets	Current
9	DR floor accounting	DR_Floor_vCPU, effective ceiling, headroom	Updated
10	Environment quota scoring	Quota_Score_{Prod,NonProd,DR}	Current
11	Capacity forecast / CR sizing	Forecast_Quantity	Current
12	Auto-increase trigger	Utilisation thresholds + debounce	Current
13	Emergency transfer & tier escalation	Tier 1/2/3, quota-neutral transfer	Current
14	Scaling & API-budget model	CRG cardinality, request-per-cycle	Current
15	DR ratio parameters (fixed %)	dr_ratio_min/max/ target	Suggested by Scenario 17
16	Reservation target & reconciliation floor	Allocated VM Count + Buffer (CAP-003)	New
17	DR destination requirement — max-not-sum	MAX(source portions per destination) (A.6)	New
18	Standby activation (DR-019)	associated → allocated staged acquisition	New
19	Deployment readiness gate	READY / QUOTA_DEFICIT / STALE_STATE / ... (RDY-002)	New
20	Customer placement seed record	First-placement persistence + reuse policy	New

Shared Building Blocks (used by several scenarios)

Default scoring weights [Assumed]

```
alpha ( ) = 0.30    # capacity / headroom signal
beta  ( ) = 0.20    # quota headroom signal
gamma ( ) = 0.25    # capacity-weighted distribution
delta ( ) = 0.15    # DR-coverage / overflow-integrity signal
epsilon()= 0.10     # zone diversity
Clamp(x) = max(0, min(1, x))      # every component clamped to [0,1] before weighting
```

All five weights are tunable policy constants stored in `PlacementPolicy` (config-as-code, versioned). They sum to exactly 1.0 and apply across all three environment scoring formulas. [Assumed]

Policy weight-sum validation rule [Decided]

The weight-sum invariant is a **mathematical requirement** for the scoring formulas to produce meaningful, comparable scores in the range $[0, 1]$. It must be enforced by a validation gate in the Config/Scope-File Service, not left to operator judgement.

Why 1.0 is required: Every component is clamped to $[0, 1]$. When weights sum to exactly 1.0, the best possible score is 1.0 and the worst is 0.0. When weights sum to any other value:

Weights sum to	Effect
> 1.0 (e.g. 1.10)	Best possible score > 1.0 — scores inflated; placement thresholds miscalibrated
< 1.0 (e.g. 0.90)	Best possible score < 1.0 — scores compressed; placement thresholds miscalibrated
Either direction	argmax still selects a winner but absolute score values are no longer meaningful; cross-version score comparisons are invalid

Formal validation rule:

VALIDATION RULE: `PlacementPolicy.WeightSumInvariant`

Fires: at policy version load time, before the version is published to the Placement Engine

Actor: Config / Scope-File Service

```
weight_sum = policy.alpha + policy.beta + policy.gamma + policy.delta + policy.epsilon
tolerance   = 0.001    # absorbs IEEE 754 floating-point representation error only;
                    # does NOT tolerate intentional deviations 0.01
```

```

IF |weight_sum - 1.0| > tolerance:
    REJECT policy version (do not publish)
    RAISE PolicyValidationError(
        code      = "WEIGHT_SUM_INVARIANT_VIOLATED",
        message   = f"PlacementPolicy weights must sum to 1.0 ± {tolerance}. "
                    f"Actual sum: {weight_sum:.6f}. "
                    f"Adjust alpha/beta/gamma/delta/epsilon before resubmitting.",
        policy_version = policy.version,
        computed_sum   = weight_sum
    )
    WRITE AuditEvent(
        event_type    = "POLICY_VALIDATION_FAILED",
        severity      = "ERROR",
        policy_version = policy.version,
        detail        = PolicyValidationError
    )
    RETURN # placement engine never receives this policy version

ELSE:
    PUBLISH policy version to Placement Engine
    WRITE AuditEvent(
        event_type    = "POLICY_VERSION_PUBLISHED",
        policy_version = policy.version,
        weight_sum     = weight_sum
    )

```

Tuning constraint — the pie rule: When an operator adjusts any weight, the total reduction from other weights must equal the total increase. Example:

```

Valid:    =0.40, =0.15, =0.25, =0.10, =0.10 → sum = 1.00
Invalid:  =0.40, =0.20, =0.25, =0.15, =0.10 → sum = 1.10 (REJECTED)
Invalid:  =0.20, =0.20, =0.25, =0.15, =0.10 → sum = 0.90 (REJECTED)

```

Required test cases:

Test ID	Input	Expected outcome
T-WV-01	=0.30, =0.20, =0.25, =0.15, =0.10 (sum=1.00)	Policy published
T-WV-02	=0.40, =0.20, =0.25, =0.15, =0.10 (sum=1.10)	WEIGHT_SUM_INVARIANT_VIOLATED, policy rejected
T-WV-03	=0.20, =0.20, =0.25, =0.15, =0.10 (sum=0.90)	WEIGHT_SUM_INVARIANT_VIOLATED, policy rejected

Test ID	Input	Expected outcome
T-WV-04	IEEE 754 rounding artefact (sum=1.0000001)	Within tolerance → policy published
T-WV-05	sum=1.01	Outside tolerance → policy rejected

[Decided]

Named regional-snapshot quantities [Decided]

```

nonprod_crg.effective_free = nonprod_crg.free_slots - dr_overflow_reserve
dr_crg.coverage_ratio      = dr_crg.quantity / potential_dr_demand
potential_dr_demand(region)= Σ prod_allocated for all customers whose dr_region = region

```

Key formula disambiguation

Formula	Purpose	Source
Target Reserved Capacity = Allocated + Buffer	Continuous reconciliation floor (Scenario 16)	CAP-003
Forecast_Quantity = ceil(Peak × (1+Growth) + DR_Buffer)	Proactive growth ahead of demand (Scenario 11)	ADR-004
Destination DR Requirement(d) = MAX(source portions)	DR standby sizing (Scenario 17)	A.6 / DR-017
dr_ratio_* constants	Superseded (Scenario 15) — configuration reference only	PRR Section 32

Scenario 1 — Prod Region Derivation (Customer Chooses a Geography)

Trigger: customer supplies a geography (e.g. “Europe”), not a specific region.
Status: Exception path — not the default (PLC-002). Requires explicit exception approval and customer acknowledgement that the **derived production region becomes fixed until an approved migration changes the seed**. Cannot be invoked without a recorded exception reference. [Decided]

Logic:

1. Build candidate set = **Standard Capacity Regions** in the chosen geography. Restricted regions are excluded before scoring. [Decided]
2. Score every candidate with **PS_Prod** from the versioned regional snapshot.
3. Select **derived Prod anchor** = $\text{argmax}(\text{PS_Prod})$.
4. **Tie-break**: deterministic — first region in the Standard Capacity Region list order for that geography; also the cold-start default when no snapshot exists. [Derived]
5. Run sequential CVAL $\rightarrow$ DR selection (Scenarios 5 & 6) on the same Standard region pool.

PS_Prod(r) [Decided]

```

PS_Prod(r) =
    0.30 × Clamp(nonprod_crg.effective_free / prod_crg.quantity)      ← : NonProd headroom s
+ 0.20 × Clamp(prod_crg.quota_headroom / prod_crg.quota_limit)      ← : Prod quota headroo
+ 0.25 × Clamp(1 - prod_customer_count / total_customers)          ← : distribution fairn
+ 0.15 × Clamp(dr_crg.coverage_ratio)                                ← : DR readiness signa
+ 0.10 × Clamp(az_count / 3)                                          ← : zone diversity

```

PS_Prod is **dual-purpose**: derives the Prod region in Scenario 1 and is reused for post-selection validation in Scenario 2. Every candidate score and the policy version are written to the **OperationRecord** for deterministic replay. [Derived]

Scenario 2 — Prod Region Validation (Customer Supplies a Specific Region)

Trigger: customer names an exact Azure region — the **default** input mode (PLC-001). ACRME **validates** the supplied region against HC-1..HC-10 and placement-policy rules; it does **not** derive a region from geography. [Decided]

Logic:

- **Standard Capacity Region**: validate against HC-1..HC-10 (Scenario 7). If eligible, it becomes the Prod anchor directly. **PS_Prod** computed for post-selection validation/audit only, not for selection.
- **Restricted Capacity Region**: do not score — route to the Exception Deployment Workflow (Scenario 3).

There is no **argmax** here. The customer's choice is authoritative once it passes the hard constraints. The first approved placement writes the **Customer Seed Record** (Scenario 20). [Decided]

Scenario 3 — Restricted Region Request (Exception Path)

Trigger: the requested or only feasible region is a **Restricted Capacity Region**.

Logic: no scoring formula is evaluated. The request diverts to the manual Exception Deployment Workflow (governance approval, EC-1..EC-4, VR-1..VR-11, HC-9/HC-10 controls). Restricted regions never enter the scoring pipeline. [Decided]

Exception conditions:

Code	Condition
EC-1	Customer requires the specific restricted region by contract
EC-2	No Standard region in the geography can supply sufficient capacity
EC-3	Data-residency requirement binds to the restricted region
EC-4	Scenario 2 input — restricted regions can never be engine-derived

Scenario 4 — Middle East Three-Region Deployment (Current position: DR_NOT_OFFERED)

Trigger: a Middle East deployment requiring Prod + CVAL (and, subject to legal, DR).

Current legal position — no DR is offered in the Middle East (DR-014, DEC-001). The Middle East programme is legal-owned and serves government/medical customers under data-residency/sovereignty laws; cross-border DR cannot meet residency requirements. The geography is therefore flagged **DR_NOT_OFFERED = true today**, and the DR offering is **under legal review (DEC-001, pending)**. Middle East **production may still be placed and governed** — DR simply does not exist for it. The Switzerland North cross-geo path below is **pre-configured but inactive**, and activates **only if and when Legal clears DEC-001** (a config flip `dr_not_offered["Middle East"] = false`, no code change).

Logic — default (current position, DR_NOT_OFFERED = true):

1. Candidate in-geo Standard regions = { Saudi Arabia Central, UAE North } (both Standard)
2. Score both with PS_Prod.
3. Prod = argmax(PS_Prod) over the two.
4. CVAL = the remaining in-geo region (deterministic - only one candidate left).

5. DR = BYPASSED. Because DR_NOT_OFFERED = true for the geography, no DR region is derived. The seed record records dr_region = NOT_OFFERED. No SourceDestinationDRIndex entry is created, no DR earmark, and no cross-geo activation are created.

DR_NOT_OFFERED is evaluated **before** any cross-geo extension: the engine never reaches the Switzerland North path while the flag is true. [Documented]

Logic — conditional (only if Legal clears DEC-001, DR_NOT_OFFERED = false):

5'. DR = cross-geo extension region: Switzerland North (Europe)
because no third in-geo Standard region exists to satisfy region separation.

v2.2 correction (REG-002): In the conditional path, the cross-geo DR extension region was incorrectly cited as “Belgium Central” in earlier material. The authoritative placement configuration specifies **Switzerland North** as the cross-geo extension for the Middle East. All references to Belgium Central are superseded by Switzerland North. This correction concerns *which* region would be used **if** DR is ever approved; it does **not** imply DR is currently offered.

Cross-geo extension constraints from ADR-001 apply to the DR region (Switzerland North) **when and if the conditional path is activated**. [Decided]

Scenario 5 — CVAL / NonProd Region Selection

Trigger: sequential step after the Prod anchor is fixed. Selects `argmax(PS_NonProd)` over eligible Standard regions.

PS_NonProd(r) — design-of-record [Decided]

PS_NonProd(r) =

0.30 × Clamp(nonprod_crg.effective_free / nonprod_crg.quantity)	← : effective NonProd
+ 0.20 × Clamp(nonprod_crg.quota_headroom / nonprod_crg.quota_limit)	← : NonProd quota
+ 0.25 × Clamp(1 - nonprod_customer_count / total_customers)	← : distribution f
+ 0.15 × Clamp(nonprod_crg.effective_free / nonprod_crg.quantity)	← : overflow capac
+ 0.10 × Clamp(az_count / 3)	← : zone diversity

Corrected pilot variant [Undocumented - architectural judgement]

The design-of-record above duplicates the same signal under `Capacity` and `Quota` (combined weight 0.45). The PRR’s corrected pilot formula removes the duplication:

PS_NonProd = 0.35 × Capacity + 0.25 × Quota + 0.25 × Distribution
+ 0.05 × DR_Overflow_Integrity + 0.10 × Zones

Distribution = 1 - Region_Assigned_Demand / Total_Assigned_Demand

Both variants are recorded. The design-of-record formula is the approved baseline; the pilot variant is the reviewer-recommended refinement. Neither is empirically validated yet.

CVAL/DR co-location rule (PLC-010): a customer’s CVAL and DR *may* co-locate in the same destination region. When co-located, CVAL capacity earmarked for DR activation must **not** be double-counted as both live CVAL headroom and available DR headroom. The `CVALEarmarkRecord` tracks this. [Decided]

Scenario 6 — DR Region Selection

Trigger: final sequential step. Selects `argmax(PS_DR)` over eligible Standard regions (or the cross-geo region for the Middle East **only** if its `DR_NOT_OFFERED` flag is **false** — see Scenario 4; while the flag is **true** (current legal position, DEC-001) this step is skipped and `dr_region = NOT_OFFERED`).

`PS_DR(r)` [Decided]

`PS_DR(r) =`

<code>0.30 × Clamp(dr_crg.free_slots / dr_crg.quantity)</code>	<code>← : DR CRG headroom</code>
<code>+ 0.20 × Clamp(dr_crg.quota_headroom / dr_crg.quota_limit)</code>	<code>← : DR quota headroom</code>
<code>+ 0.25 × Clamp(1 - dr_customer_count / total_customers)</code>	<code>← : distribution fairness</code>
<code>+ 0.15 × min(1.0, dr_crg.coverage_ratio / dr_ratio_target)</code>	<code>← : coverage-ratio headroom</code>
<code>+ 0.10 × Clamp(az_count / 3)</code>	<code>← : zone diversity</code>

Note on : `dr_ratio_target` in the `PS_DR` formula is now a **configurable bootstrap reference** rather than a fixed 30–40% constant. See Scenario 17 for the authoritative DR sizing formula. [Decided]

Scenario 7 — Hard-Constraint Eligibility Gate (Arithmetic Constraints)

Before any region is scored (Scenario 1) or accepted (Scenario 2), it must pass the hard constraints.

HC-3 QUOTA_FLOOR (per environment) [Decided]

Prod:	<code>Prod_Group_headroom(R)</code>	<code>requested_vm_count × vCPU</code>	
NonProd:	<code>nonprod_headroom(R)</code>	<code>requested_vm_count × vCPU</code>	<code># uses effective_nonprod_ceiling</code>
DR:	<code>dr_headroom(R)</code>	<code>target_dr_qty × vCPU</code>	

HC-6 DR_COVERAGE_FLOOR [Decided]

Region ineligible for DR if:

$(\text{dr_crg.free_slots} + \text{nonprod_crg.effective_free}) < \text{customer_requested_dr_slots}$

HC-7 DR_FLOOR_INTEGRITY [Decided]

REJECT NonProd placement in region R if:

$\text{nonprod_quota_used}(R) + (\text{requested_vm_count} \times \text{vCPU}) > \text{effective_nonprod_ceiling}(R)$

HC-3 checks raw group headroom; HC-7 additionally enforces floor compliance. Both must pass. On breach: emit DRFloorViolationDetected (Critical severity).

Minimum-headroom policy defaults [Decided]

$\text{min_prod_quota_headroom_vcpu} = 20$
 $\text{min_nonprod_quota_headroom_vcpu} = 20$
 $\text{min_dr_headroom_vcpu} = 16$

Scenario 8 — Quota Group Sizing (per Region)

Primary model (QUA-004): one governed quota pool per region and quota family, covering Prod, NonProd/CVAL, and DR together. Prod and DR are protected inside the shared pool by logical earmarks, not by physical group separation. [Decided]

$\text{Pool_Limit}(\text{region}) =$

$\text{Prod_CRG_quantity} \times \text{vCPU} \times (1 + \text{prod_growth_buffer})$
 $+ \text{NonProd_CRG_quantity} \times \text{vCPU} \times (1 + \text{nonprod_growth_buffer})$
 $+ \text{DR_Earmark_vCPU}(\text{region})$
 $+ \text{emergency_transfer_headroom_vcpu}$

$\text{prod_growth_buffer} = 0.20$

$\text{nonprod_growth_buffer} = 0.20$

$\text{DR_Earmark_vCPU}(\text{region}) = \text{Destination_DR_Requirement}(\text{region}) \times \text{vCPU}$ [max-not-sum]

$\text{emergency_transfer_headroom_vcpu} = \text{max_emergency_transfer_qty} \times \text{vCPU}$

$\text{max_emergency_transfer_qty} = \text{potential_dr_demand} \times \text{emergency_transfer_pct}$

$\text{emergency_transfer_pct} = 0.30$

Because Prod, NonProd/CVAL, and DR share one pool, an emergency DR draw needs **no cross-group transfer**: the DR orchestrator consumes Pool_Headroom first, then reclaimable NonProd above committed demand. This is the manipulation-flexibility gain that motivates the single-pool model (avoids fragmenting quota and cuts quota-increase requests to Microsoft). [Decided]

Prod protection inside the shared pool (logical earmarks, enforced at allocation/reclamation time):

```

Prod_Reserved_Floor = Prod_Used_vCPU + Prod_Growth_Buffer_vCPU
Allocatable_NonProd = Pool_Limit - Prod_Reserved_Floor - DR_Earmark_vCPU - NonProd_Used_vCPU
Pool_Headroom       = Pool_Limit - Pool_Used

```

Prod_Reserved_Floor and DR_Earmark_vCPU are never allocatable to NonProd; NonProd allocation fail-safes when Allocatable_NonProd reaches zero. [Decided]

Exception topology (only when Azure Quota Group limits or a mandatory Prod-isolation governance boundary make one pool impossible) — two groups per region, recorded in the Decision Log:

```

Prod_Group_Limit(region) =
    Prod_CRG_quantity × vCPU × (1 + prod_growth_buffer)

NonProd_DR_Group_Limit(region) =
    NonProd_CRG_quantity × vCPU × (1 + nonprod_growth_buffer)
    + DR_CRG_quantity    × vCPU
    + emergency_transfer_headroom_vcpu

```

In the exception topology the `emergency_transfer_headroom_vcpu` term is what makes Tier 2 DR expansion quota-neutral within the shared NonProd+DR group. In the single-pool model this term is subsumed by the shared `Pool_Headroom`. [Decided]

Quota-as-governor principle (QUA-005): quota allocation caps deployable capacity per product, environment, subscription, region, and VM family. Teams must justify quota increases — unallocated pooled quota is not automatically distributed. [Decided]

Scenario 9 — DR Floor Accounting (Engine-Enforced Sub-Limit) — Updated

Azure quota pools/groups have no native intra-pool sub-reservation; the engine enforces the DR floor by arithmetic (logical earmark) in both the single-pool and exception two-group topologies. [Decided]

Accounting formulas [Decided]

```

DR_Floor_vCPU(region) = Destination_DR_Requirement(region) × vCPU
                        [v2.2: uses max-not-sum formula from Scenario 17]
                        [v1: was potential_dr_demand × vCPU × dr_ratio_max -
superseded]

```

```

# Single governed pool (primary model, QUA-004):
DR_Earmark_vCPU(region) = DR_Floor_vCPU(region)      # never allocatable to NonProd
Allocatable_NonProd     = Pool_Limit - Prod_Reserved_Floor - DR_Earmark_vCPU - NonProd_Used_vCPU

```

Pool_Headroom = Pool_Limit - Pool_Used

Exception two-group topology only:

Effective_NonProd_Ceiling = NonProd_DR_Group_Limit - DR_Floor_vCPU

NonProd_Headroom = Effective_NonProd_Ceiling - NonProd_Used_vCPU

Group_Headroom = Group_Limit - Group_Used

dr_crg.coverage_ratio = dr_crg.quantity / Destination_DR_Requirement(region)

v2.2 change: DR_Floor_vCPU is no longer derived from potential_dr_demand × dr_ratio_max. It is now derived from Destination_DR_Requirement(region) — the max-not-sum value from Scenario 17. This aligns the floor with the lean bootstrap model (DR-007) and the distributed DR topology (DR-016). The fixed dr_ratio_max = 0.40 constant is retained in Scenario 15 for legacy reference only.

Dual-validation control [Decided]

A separate detector recomputes the DR floor from authoritative assignment data; any disagreement between command-time and detector values disables automatic NonProd expansion until reconciled.

Worked topology example [Decided]

Prod group = 128 vCPU (32 × D4s_v3, 4 vCPU each)

NonProdDR group = 80 vCPU

DR floor = 32 vCPU (= Destination_DR_Requirement for this region)

Effective NonProd ceiling = 48 vCPU (80 - 32)

Scenario 10 — Environment Quota Scoring (Component Detail)

The generic Quota_Score is dispatched by environment type. [Decided]

Quota_Score_Prod(r) = prod_group_headroom_vcpu / prod_group_limit_vcpu

Quota_Score_NonProd(r) = nonprod_headroom_vcpu / effective_nonprod_ceiling_vcpu

Quota_Score_DR(r) = dr_headroom_vcpu / dr_floor_vcpu

Semantics differ by type: a DR score of 1.0 means maximum expansion room (DR CRG at zero), not maximum quota consumed. Zero-denominator guards apply. [Decided]

Scenario 11 — Capacity Forecast / CR Sizing

Advisory forecast used to size CR quantity over a horizon — the **proactive growth path**, distinct from the continuous reconciliation floor (Scenario 16).
[Decided]

`Forecast_Quantity = ceil( Forecast_Peak × (1 + Growth_Buffer) + DR_Buffer )`

<code>Forecast_Peak</code>	= predicted peak allocated-VM demand in the horizon	[Derived]
<code>Growth_Buffer</code>	= policy % for forecast uncertainty	[Assumed]
<code>DR_Buffer</code>	= extra units required by approved recovery policy	[Assumed]
<code>Forecast_Horizon</code>	{ 30, 60, 90 } days	[Assumed]

Lead-time alerting: when forecast demand approaches 80% of quota limit, emit `ForecastApproachingQuotaLimit` with 14-day lead time. [Documented]

Forecast recommendations remain advisory until model accuracy and false-positive rates are measured. The horizon and buffer values are policy percentages stored in `PlacementPolicy`, not fixed constants.

Scenario 12 — Auto-Increase Trigger (Steady-State Capacity Lifecycle)

When utilisation crosses a threshold, a `CapacityIncreaseRequest` is raised.
[Decided]

Trigger auto-increase when utilisation	threshold:
<code>dr_autoincrease_threshold</code>	= 0.35
<code>prod_autoincrease_threshold</code>	= 0.20
<code>nonprod_autoincrease_threshold</code>	= 0.20

`Debounce cooldown = 30 min per (region + CRG type) after any trigger`

Phase 1 requires operator approval. Auto-decrease excluded from Phase 1. The steady-state lifecycle runs **only** in `STEADY_STATE` engine mode (see ADR-003).
[Decided]

Normative 10-step steady-state capacity lifecycle

1. Detect threshold crossing.
2. Re-read current CR, quota, sharing, and assignment state.
3. Create `CapacityIncreaseRequest`.
4. Calculate target quantity (`Forecast_Quantity` or `Allocated + Buffer` floor, whichever is higher).
5. Require operator approval (Phase 1).
6. Submit the quota action only if validated as required.
7. Wait for confirmed quota state — no assumed propagation SLA.

8. Update CR quantity.
9. Confirm the actual quantity.
10. Refresh the snapshot and close the request.

Scenario 13 — Emergency Transfer & Tier Escalation (DR Event Active)

During an active DR event (DR_EVENT_ACTIVE engine mode), emergency capacity is obtained by tier. [Decided]

Tier 1	DirectExpansion	- additive DR expansion using existing headroom. Automated.
Tier 2	QuotaNeutralTransfer	- reduce NonProd CR, expand DR within the SAME quota group. Policy-gated. Quota-neutral (see below).
Tier 3	DestructiveTransfer	- changes VM associations. Dual-approval + elevated RBAC. BLOCKED in Phase 1.

Quota-neutral math for Tier 2 [Derived - POC-31/32]

Because NonProd CR and DR CR draw from the SAME NonProd+DR quota group:

NonProd CR reduction → releases $q \times \text{vCPU}$ to the GROUP pool
 DR CR expansion → consumes $q \times \text{vCPU}$ from the SAME GROUP pool
 Net group headroom change 0 → no Azure quota-increase request needed
 Tier RTO gated only by ARM operation time (minutes), not quota approval (hours)

$\text{max_emergency_transfer_qty} = \text{potential_dr_demand} \times \text{emergency_transfer_pct}$ (pct = 0.30)

Escalation order: Tier 1 headroom available? → Tier 2 approved? → Tier 3 allowed? → manual.

Scenario 14 — Scaling & API-Budget Model

Used to size the estate and stay within Azure Resource Manager throttling budgets. [Derived]

$\text{CRG_lower_bound} = 3 \times R$ # 3 CRGs (Prod/NonProd/DR) per region
 $\text{CRG_total} = R \times \text{Eclass} \times Z \times \text{SKUset} \times \text{IsolationFactor}$

$\text{Requests_per_cycle} = \text{CRG_reads} + \text{CR_reads} + \text{sharing_reads} + \text{quota_reads} + \text{association_reads}$
 $\text{Average_requests_per_second} = \text{Requests_per_cycle} / 300$ # 5-minute (300 s) cycle

Documented ARM throttle baselines [Documented]

Compute RP read limit = 250 requests / 5 min / subscription
 Compute RP write limit = 1,200 requests / hour / subscription

The adaptive throttle manager initialises with these baselines and adapts to observed 429/Retry-After.

Reconciliation cadence [Assumed]

Reconciliation loop target	= 6 min (configurable; production interval TBD)
Critical targeted reconcile P95	< 2 min
Stable-resource state age P95	< 15 min
Drift detection	2 reconciliation cycles

Scenario 15 — DR Ratio Parameters (Fixed %) — SUPERSEDED

Status: Superseded by Scenario 17 (max-not-sum). These constants are retained for configuration reference and legacy comparison only. The fixed-percentage DR sizing model (30-40% of Prod) was rejected in Requirements v2.0/v2.1 because it produces idle reserves estimated at **\$1.5M–\$5M/year** at platform scale — directly contradicting the cost-reduction mandate. See Appendix D of the Requirements Baseline for the full derivation.

dr_ratio_min	= 0.30	[superseded - was HC-6 lower bound]
dr_ratio_max	= 0.40	[superseded - was DR_Floor_vCPU multiplier]
dr_ratio_target	= [0.30, 0.40]	[superseded - now: configurable bootstrap per product/workload]
emergency_transfer_pct	= 0.30	[still current - Scenario 13]
prod_growth_buffer	= 0.20	[still current - Scenario 8]
nonprod_growth_buffer	= 0.20	[still current - Scenario 8]

The SUM override (C-11 in the Configurable Items Register) remains available for specific customers or geographies that contractually require protection against concurrent regional failures. All other scopes use the max-not-sum formula (Scenario 17).

Scenario 16 — Reservation Target & Reconciliation Floor (CAP-003) — New

Trigger: every reconciliation cycle compares current reserved quantity against this floor. This is the **continuous floor formula** — distinct from the forecast-based growth formula (Scenario 11).

A.1 Reservation target (CAP-003) [Decided]

Target Reserved Capacity = Allocated VM Count + Configured Buffer

- **Allocated VM Count** = VMs in running / allocated state consuming compute capacity.
- **Configured Buffer** = policy-defined headroom above allocated demand. Tunable per product/region/env/SKU. Not hard-coded.
- **Associated-but-deallocated VMs** do not force reservation retention (CAP-004). They are reported separately so teams understand restart risk, but do not add to the target.

A.2 Reservation headroom

Reservation Headroom = Reserved Quantity - Allocated VM Count

A.3 Reservation deficit

Reservation Deficit = max(0, Target Reserved Capacity - Reserved Quantity)

Reconciliation decision logic [Decided]

IF Reserved Quantity < Target Reserved Capacity:

 RAISE reservation (scale-up) toward target, subject to Azure availability.

 IF Azure cannot supply: hold current state, RAISE alert, expose buffer deficit.

IF Reserved Quantity > Target Reserved Capacity (consistently, after minimum-hold interval)

 LOWER reservation toward target - ONLY when all guards pass:

- current_reserved > allocated_count (never below allocated)
- not within DR protection window
- no active maintenance exclusion
- cost policy permits reduction

Zero-capacity and no-auto-delete policy (CAP-009/010) [Decided]

Reduce an unused managed reservation to ZERO; do NOT delete the reservation object.

Normal reconciliation NEVER deletes CRGs or reservation definitions.

Deletion uses a separate approved decommissioning workflow only.

Scenario 17 — DR Destination Requirement — Max-Not-Sum (A.6/DR-017) — New

This scenario replaces the fixed `dr_ratio_*` sizing in Scenario 15.

The single-failure assumption (DR-001) [Decided]

The programme plans for failure of **one** production region within a geography at a time. Simultaneous multi-region failure is outside the default guaranteed model.

A.6 DR destination requirement — corrected formula [Decided]

`Destination_DR_Requirement(d)`
= MAX over each non-concurrent source region `s` protected by `d` (
 `Workload_Portion(s → d)`
)

Where: - `d` = destination region hosting DR standby capacity - `s` = source region whose production customers have DR in `d` - `Workload_Portion(s → d)` = the quantity of VMs/vCPUs from source `s` that are assigned to land in destination `d` on failover (from the source→destination DR index, DR-018)

Rationale: because only one region fails at a time, destination `d` never simultaneously hosts failover from more than one source. It therefore needs standby capacity for its **largest** protected source only — never the sum of all sources. The standby slots are **shared** / **overcommitted** across mutually exclusive failure events. [Decided]

A.7 DR capacity gap [Decided]

`DR_Capacity_Gap(d)` = max(0,
 `Destination_DR_Requirement(d)` - `Usable_Destination_Capacity(d)`
)

`Usable_Destination_Capacity(d)` may include approved bootstrap headroom, available CRG reservations, releasable CVAL capacity (after earmark check, Scenario 9/PLC-010), and capacity acquired through approved sharing or expansion.

A.8 Shared-DR overcommit ratio (informational) [Derived]

`Overcommit_Ratio(d)` = SUM(`Workload_Portion(s → d)` for all `s`) / MAX(`Workload_Portion(s → d)`)

A ratio > 1 quantifies: (a) the capacity and cost saved by max-not-sum sizing, and (b) the exposure if the single-failure assumption is ever violated. Required input for POC-011 risk sign-off.

Observability requirement (OBS-002) [Decided]

Alert when a destination's usable capacity falls below `Destination_DR_Requirement(d)` — the max protected source — not below the sum. Dashboard must show per-destination max-source coverage (OBS-004).

Conservative SUM override (C-11) [Decided]

`Destination_DR_Requirement_Conservative(d)` = SUM(`Workload_Portion(s → d)` for all `s`)

Used only where a customer or geography contract explicitly requires protection against concurrent regional failures. Configured per-scope in `PlacementPolicy`; no code change required.

Worked example (from Requirements Appendix D) [Decided]

Region 2 holds DR standby for customers from Region 1 and Region 3:

Source protected by R2	Failover portion in R2
Region 1 (Cust1 + Cust5)	120 cores
Region 3 (Cust6)	80 cores

SUM sizing (old): Requirement(R2) = 120 + 80 = 200 cores + over-provisions; superseded

MAX sizing (new): Requirement(R2) = max(120, 80) = 120 cores

Saving at R2: 200 → 120 = 80 cores (40%) removed with no loss of protection.

Overcommit_Ratio(R2) = 200 / 120 = 1.67

→ R2's shared standby is 1.67× oversubscribed.

→ Platform-wide extrapolation: ~40-50% standby reduction \$0.6M-\$2.5M/year saved.

Source→Destination DR Index entity (DR-018) [Decided]

The max-not-sum formula depends on an authoritative bidirectional mapping
— a required state-store entity:

```
SourceDestinationDRIndex {
    source_region          : string
    destination_region     : string
    customer_realm_id      : string
    standby_instance_set   : list<InstanceRef>
    sku_family             : string
    quantity_vcpu          : int
    activation_state        : STANDBY | ACTIVATING | ACTIVE | FAILBACK_PENDING
    last_updated           : datetime
    policy_version         : string
}
```

This index is the reverse view of the customer seed record (Scenario 20). On a regional failure, the engine queries this index to determine exactly which standby sets to activate and where. [Decided]

Scenario 18 — Standby Activation (DR-019) — New

Trigger: authorised DR declaration; engine mode transitions to DR_EVENT_ACTIVE.

Per-customer activation workflow [Decided]

Each customer's DR instances transition associated → allocated (standby → active) in approved business-priority waves:

Priority Wave 0 (P0): Platform control planes and recovery orchestration infrastructure.
Priority Wave 1 (P-1): Highest-priority customer workloads.
Priority Wave N: Remaining customers in business-priority order.

DR-006 staged capacity acquisition per wave:

Stage 1: Use approved bootstrap / pre-staged headroom (zero Azure wait time).
Stage 2: Allocate available reservation + quota already in the destination.
Stage 3: Shut down / disassociate eligible CVAL workloads (after DR-010 authorised trigger).
Stage 4: Share or reassign reservations within supported region/zone boundaries.
Stage 5: Allocate pooled quota to DR subscriptions.
Stage 6: Request additional Azure quota/capacity where required.
Stage 7: Report unrecoverable capacity gaps.

Stages 1–2 start immediately from bootstrap headroom — P0/P-1 customers begin recovery without waiting on Azure deallocation/disassociation APIs (which are throttled during regional events). [Decided]

Activation state tracking [Decided]

```
ActivationRecord {
    customer_realm_id : string
    source_region      : string
    destination_region : string
    priority_wave      : int
    activation_state    : PENDING | ACTIVATING | ACTIVE | FAILED | FAILBACK_PENDING
    stage_reached      : int          # 1-7 per DR-006
    capacity_gap_vcpu   : int          # 0 if fully provisioned
    activated_at        : datetime
    approved_by        : string
    incident_id         : string
}
```

Activation state is tracked per customer, auditable, and reversible on failback (DR-013). [Decided]

Failback reversal [Decided]

On failback: ACTIVE → FAILBACK_PENDING → STANDBY in reverse priority order. The seed record and DR index are preserved; no re-derivation of placement. History, audit trail, and DR capacity state are retained through the full failback cycle.

Scenario 19 — Deployment Readiness Gate (RDY-002) — New

Trigger: every AEP-triggered deployment before the first environment is provisioned.

Machine-readable readiness states [Decided]

READY	All gates pass; deployment may proceed.
READY_WITH_RISK	All gates pass but one or more advisory risks detected (e.g. buffer b
QUOTA_DEFICIT	Required deployment quota > available quota in the deploying subscript
RESERVATION_DEFICIT	Reservation exists but quantity insufficient; approved over-allocation
CAPACITY_UNAVAILABLE	Azure cannot supply the requested SKU/zone at this time.
STALE_STATE	Capacity/quota snapshot exceeds max-age threshold; cannot drive a safe
POLICY_BLOCKED	A governance policy (hard constraint, exception, approval) blocks the
VALIDATION_REQUIRED	One or more validation steps (e.g., POC-gated behaviours) not yet cont

Readiness gate logic (RDY-001) [Decided]

All of the following must pass for READY:

1. target_region approved_region_catalogue (REG-001)
2. az_count 1 in target_region (CAP-011)
3. sku supported_skus(target_region)
4. reservation_policy known for (subscription, region, zone, sku)
5. reservation exists (if required by policy)
6. reserved_quantity requested_count OR approved_over_allocation active
7. consumer_subscription_quota requested_units (QUA-013 - POC-gated)
8. snapshot_age max_snapshot_age_seconds (RDY-004)
9. no blocking policy exception or hard constraint violation

Staleness check (RDY-004) [Decided]

```
IF now() - snapshot.collected_at > max_snapshot_age:
    RETURN STALE_STATE
    ACTION: refresh synchronously OR block with STALE_STATE -
    never drive placement from stale data
```

Quota deficit formula (A.5) [Decided]

```
Quota_Deficit = max(0, Requested_Deployment_Units - Available_Quota)
Available_Quota = Assigned_Regional_VM_Family_Quota - Current_Regional_VM_Family_Usage (A.
```

Scenario 20 — Customer Placement Seed Record (PLC-003/004/005) — New

Trigger: first approved production-region placement for a customer in a geography.

Seed record creation [Decided]

```
CustomerSeedRecord {
    customer_realm_id      : string      # authoritative customer/realm identifier
    geography              : string      # e.g. "NorthAmerica", "Europe"
    production_region      : string      # exact Azure region name
    cval_region            : string      # exact Azure region name
    dr_region              : string | "NOT_OFFERED"
    products_covered       : list<string> # all products seeded by this record
    decision_timestamp     : datetime
    policy_version         : string      # PlacementPolicy version used
    capacity_snapshot_ref  : string      # snapshot ID that drove the decision
    exception_ref          : string | null # approval ID if Restricted region or geo-exception
    approval_metadata      : ApprovalRecord
    input_mode             : "SPECIFIC_REGION" | "GEOGRAPHY_EXCEPTION"
}
```

Seed reuse policy (PLC-004) [Decided]

```
IF CustomerSeedRecord exists for (customer_realm_id, geography):
    READ seed record - do NOT re-invoke the placement engine.
    Subsequent products and environments use the seeded production/CVAL/DR regions.
    Engine is NOT re-run per product once seeded.
```

Controlled seed change (PLC-005) [Decided]

```
IF seed change requested:
    REQUIRE approved migration/exception workflow.
    REQUIRE impact analysis across all products covered by the seed.
    PROHIBIT automatic regeneration on: upgrades, rebuilds, routine deployments.
```

A. Core Formula Reference (Appendix A — Requirements v2.1)

Label	Formula	Source
A.1	Target Reserved Capacity = Allocated VM Count + Configured Buffer	CAP-003
A.2	Reservation Headroom = Reserved Quantity - Allocated VM Count	CAP-003
A.3	Reservation Deficit = max(0, Target Reserved Capacity - Reserved Quantity)	CAP-003
A.4	Available Quota = Assigned Regional VM-Family Quota - Current Usage	QUA-002
A.5	Quota Deficit = max(0, Requested Deployment Units - Available Quota)	QUA-007
A.6	Destination_DR_Requirement(d) = MAX over s (Workload_Portion(s → d))	DR-017
A.7	DR_Capacity_Gap(d) = max(0, DR_Requirement(d) - Usable_Capacity(d))	DR-017
A.8	Overcommit_Ratio(d) = SUM(portions) / MAX(portions)	DR-017

B. Consolidated Policy-Constant Table (v2.2)

Constant	Value	Used in Scenario(s)	Status
alpha / beta / gamma / delta / epsilon	0.30 / 0.20 / 0.25 / 0.15 / 0.10	1, 5, 6	Current
prod_growth_buffer	0.20	8	Current
nonprod_growth_buffer	0.20	8	Current

Constant	Value	Used in Scenario(s)	Status
emergency_transfer_pct	0.30	8, 13	Current
min_prod_quota_headroom_vcpu	20	7	Current
min_nonprod_quota_headroom_vcpu	20	7	Current
min_dr_headroom_vcpu	16	7	Current
dr_autoincrease_threshold	0.35	12	Current
prod/nonprod_autoincrease_threshold	0.40	12	Current
Debounce	30 min	12	Current
cooldown			
Reconciliation	6 min	14	Current
loop target	(configurable)		
ARM read / write baseline	250 per 5 min / 1,200 per hour	14	Current
Forecast_Horizon	30 / 60 / 90 days	11	Current
dr_ratio_min	0.30	15	Superseded
dr_ratio_max	0.40	15	Superseded
dr_ratio_target	[0.30, 0.40]	15	Superseded
DR bootstrap target	Configurable per prod-uct/workload — no fixed %	17	Replaces ratio
Max-not-sum default	MAX(source portions)	17	Current
SUM override (C-11)	SUM(source portions) — per-scope opt-in	17	Current
EU cross-geo DR extension region (Middle East)	Switzerland North — pre-configured but inactive ; conditional on DEC-001 (current position DR_NOT_OFFERED)	4	Updated (was Belgium Central; now gated by DEC-001)

C. Evidence & Maturity Summary

Formula	Evidence level	Validation required
PS_Prod, PS_NonProd, PS_DR (scoring)	[Decided] — design-of-record	Shadow/recommendation mode until empirically validated
HC-3, HC-6, HC-7 (hard constraints)	[Decided]	Ready for Phase 1
DR floor accounting (Scenario 9)	[Decided]	Updated to use max-not-sum input
Tier 2 quota-neutral math (Scenario 13)	[Derived]	POC-31/POC-32 required
Max-not-sum DR sizing (Scenario 17)	[Decided]	POC-011 required before production dependency
Consumer-subscription quota (QUA-013)	[Assumed]	POC-001 — top technical unknown
Reservation target floor (Scenario 16)	[Decided]	Ready for Phase 1
Standby activation staging (Scenario 18)	[Decided]	Dependent on POC-005 (VM state semantics)
Deployment readiness gate (Scenario 19)	[Decided]	Ready for Phase 1
Customer seed record (Scenario 20)	[Decided]	Ready for Phase 3

All constants are policy defaults stored in `PlacementPolicy` (config-as-code, versioned). Tuning any constant requires updating the config; no code change is needed. The `dr_ratio_*` constants are retained in the codebase as fallback references for the SUM override (C-11) only.

Document version 2.2 — 27 August 2026. Next review: upon POC-001 / POC-011 results.